

MediRun

ROS 2 소프트웨어 · 핵심 노드 · 데이터 로그 설계

노드 구성 · 인터페이스 · 상세설계 · 상태 머신 · 데이터 모델

TurtleBot3 Burger · ROS 2 Humble · Nav2 · Arduino · Qt(C++) · PostgreSQL

PROJECT MEMBERS

도희정 • 최치언

2026.07.22 | SOFTWARE DESIGN v2

1. ROS 2 소프트웨어 구성

문서 관계

자동 실행 환경(PM2)과 프로세스 목록은 요구사항·설계 명세서 §11로 이관했다. 이 문서는 실행된 노드들이 무엇을 주고받고 각자 무엇을 책임지는지를 다룬다. 네이밍 규칙은 개발 규칙서 §4를 따른다.

1.1 노드 목록

노드	책임	구분
`emergency_dispatcher`	SOS ID 검증, 중복 제거, 웨이포인트 조회, 미션 요청	필수
`mission_manager`	미션 상태·우선순위·Nav2 액션·재시도·복귀·목표 보관	필수
`waypoint_manager`	YAML 로드, map Pose 검증, 이름/버튼 조회	필수
`safety_manager`	속도 차단, 자동 재출발 판정, 속도 제한, 신규 목표 차단	필수
`payload_monitor`	홀센서 기반 AED 적재·반출 판정, 복귀 트리거	필수
`sos_bridge`	버튼 모듈 시리얼 수신, 디바운스, SOS 이벤트 발행	필수
`arduino_bridge`	초음파 거리, 홀센서 상태, LED 제어	필수
`audio_manager`	출동·경고 반복·도착·119 안내·정지 음성	필수
`led_manager`	상태별 LED 모드 결정(경광 모드 포함)	필수
`docking_manager`	ArUco 정렬·접근·도킹 판정·재시도	필수
`telemetry_bridge`	위치·배터리·미션·센서·진단을 대시보드용 토픽으로 정리	필수
`event_logger`	로봇 내부 DB 기록 (1차 저장, 동기화 여부와 무관)	필수
`network_monitor`	heartbeat, 단절 감지, 자동 재연결, 통합 DB 동기화	필수
`log_retention`	1일 주기 기록·파일 정리	필수
`remote_control_bridge`	관리자 수동 제어, 권한 전환	권장
`camera_ros` (`turtlebot3_bringup`, 외부)	CSI 카메라(IMX219) 입력 발행. medirun 자체 노드가 아니며 turtlebot3_bringup 소유. 상하 반전 보정은 GUI(RosCameraFeedProvider)에서 처리	보조
cartographer_node / cartographer_occupancy_grid_node (cartographer_ros, 외부)	SLAM 매핑(occupancy grid 생성). medirun 자체 노드가 아니며 turtlebot3_cartographer 소유. medirun_gui가 Ubuntu(GUI 실행 PC)에서 직접 launch하며, 로봇(Pi)에서는 실행하지 않는다 (Pi bringup의 start_cartographer는 기본값 false)	보조

1.2 책임 분리 원칙

노드	경계
`sos_bridge`	시리얼 프레임 해석까지만 담당한다. 호출의 타당성은 판단하지 않는다
`emergency_dispatcher`	호출을 해석하지만 Nav2를 직접 제어하지 않는다
`mission_manager`	자율주행 액션의 생성·취소·재시도를 단독으로 소유하며, 급정지 중에도 목표 Pose를 보관한다
`safety_manager`	최상위 우선순위로 `/cmd_vel` 통과 여부를 결정하고 자동 재출발을 단독 판정한다
`docking_manager`	최종 접근 구간에서만 `/cmd_vel`을 직접 발행한다
`event_logger`	로컬 기록만 책임진다. 통합 DB 전송은 `network_monitor`의 몫이다
Qt 대시보드	ROS 2 토픽을 구독해 표시할 뿐 로봇의 로컬 안전 루프를 대체하지 않는다

1.3 시작 전 선행 조건

검사	정상 조건	실패 처리
Bringup/LiDAR/odom	토픽 수신 및 diagnostics 정상	ERROR, 미션 거부
map/TF/AMCL	`map→base_link` 변환과 pose 유효	LOCALIZATION_REQUIRED
Nav2 lifecycle	`navigate_to_pose` 액션 서버 active	NAV2_UNAVAILABLE
waypoint_map	파일·필수 키·중복 ID·Pose 범위 정상	노드 초기화 실패
버튼 시리얼	버튼 모듈 포트 연결, `READY` 수신	SOS_BRIDGE_DOWN, 호출 접수 불가
센서 시리얼	`/dev/ttyACM*` 연결, 초음파·홀센서 수신	SENSOR_BRIDGE_DOWN
AED 적재	홀센서 `HALL:1`	PAYLOAD_MISSING, 미션 거부
배터리	30% 이상	LOW_BATTERY
로컬 DB	로컬 PostgreSQL 접속 정상	DB_UNAVAILABLE, 파일 임시 기록
통합 DB	접속 정상 (선택)	미동기 누적, **미션 수행에는 영향 없음**
DDS 구현체	로봇(Pi)·GUI 실행 PC 모두 RMW_IMPLEMENTATION=rmw_cyclonedds_cpp로 통일	Fast-DDS 혼용 시 Cartographer 내부 상태 토픽 드롭, /map 퍼블리셔 중복 인식 등 (TS-SLAM-02)

2. ROS 2 인터페이스 명세

2.1 핵심 토픽

ID	토픽	자료형	방향	주기/QoS
IF-001	`/medirun/sos/event`	`SosEvent`	sos_bridge→dispatcher	이벤트 / Reliable
IF-002	`/medirun/mission/request`	`MissionRequest`	dispatcher→manager	이벤트 / Reliable
IF-003	`/medirun/mission/status`	`MissionStatus`	manager→관제	2Hz / Reliable + Transient
IF-004	`/medirun/emergency_stop`	`Bool`	GUI→safety	이벤트 / Reliable + Transient
IF-005	`/medirun/safety/status`	`SafetyStatus`	safety→전체	10Hz / Reliable + Transient
IF-006	`/medirun/payload/state`	`PayloadState`	payload_monitor→전체	상태 변화 시 / Reliable + Transient
IF-007	`/medirun/system/event`	`SystemEvent`	전체→logger/GUI	이벤트 / Reliable
IF-008	`/arduino/button`	`String`	버튼 모듈→sos_bridge	이벤트 / Reliable
IF-009	`/arduino/ultrasonic`	`Range`	Arduino→safety	10Hz / Best effort
IF-010	`/arduino/hall`	`Bool`	Arduino→payload_monitor	10Hz / Reliable
IF-011	`/arduino/led`	`String`	led_manager→Arduino	이벤트 / Reliable
IF-012	`/medirun/audio/say`	`String`	전체→audio_manager	이벤트 / Reliable

ID	토픽	자료형	방향	주기/QoS
IF-013	`/aruco/markers`	`String` (JSON)	aruco→docking	10Hz / Best effort
IF-014	`/amcl_pose`	`PoseWithCovarianceStamped`	AMCL→관제	5Hz(갱신 시) / Reliable
IF-015	`/battery_state`	`BatteryState`	driver→전체	1Hz / Reliable
IF-016	`/scan`	`LaserScan`	LiDAR→Nav2	10Hz / Best effort
IF-017	`/camera/image_raw/compressed`	`CompressedImage`	camera→aruco/GUI	10~15FPS / Best effort
IF-018	`/cmd_vel`	`Twist`	Nav2/docking→safety→base	필요 시 / Reliable
IF-019	`/medirun/network/status`	`SystemEvent`	network_monitor→전체	1Hz / Reliable
IF-020	`/initialpose`	`PoseWithCovarianceStamped`	GUI→AMCL	이벤트 / Reliable
IF-021	`/speed_limit`	`nav2_msgs/SpeedLimit`	GUI→Nav2 controller_server	이벤트 / Reliable

속도 명령 경로: `/cmd_vel`은 `safety_manager`를 반드시 경유한다. 급정지 상태에서는 상위 명령을 통과시키지 않고 0속도를 발행한다.

2.2 서비스 · 액션

구분	이름	목적	결과
Service	`/medirun/mission/cancel`	현재 미션 취소	success, code, message
Service	`/medirun/return_home`	홈 복귀 요청	success, code, message
Service	`/medirun/dock/start`	도킹 시퀀스 시작	success, code, message
Service	`/medirun/safety/estop`	대시보드 비상정지 설정·해제	success, code, message
Service	`/medirun/log/sync_now`	미동기 기록 즉시 동기화(어드민)	synced_rows, pending_rows

Service	`/medirun/log/retention_run`	보관 정책 즉시 실행(어드민)	deleted_rows, pending_rows

구분	이름	목적	결과
Action	/navigate_to_pose	Nav2 map Pose 이동	result, feedback
Action	/follow_waypoints	웨이포인트관리 페이지 경로 시험주행(미리보기) — 지점마다 정지 후 재출발	result, feedback
Action	/navigate_through_poses	미션 출동·복귀 시 버튼에 연결된 경로(trajectory)를 저장 순서 그대로 중간 정지 없이 이어서 주행	result, feedback
Action	/medirun/execute_delivery	호출~도착~복귀 상위 미션	final_state, duration, error

2.3 사용자 정의 메시지 최소 필드

메시지	필드
SosEvent.msg	Header header; string event_id; string button_id; string source; bool pressed
MissionRequest.msg	string mission_id; string button_id; string destination_key; PoseStamped target; uint8 priority
MissionStatus.msg	string mission_id; string state; string destination; float32 distance_remaining; PoseStamped held_goal; string error_code
SafetyStatus.msg	bool motion_allowed; bool emergency_stop; string stop_reason; bool auto_resume_ready; string[] blocking_conditions; Time updated_at
PayloadState.msg	bool aed_present; Time changed_at; uint16 stable_ms
SystemEvent.msg	string event_id; string mission_id; uint8 severity; string event_type; string message; bool synced

AURIX

3. 핵심 노드 상세설계

3.1 emergency_dispatcher

항목	정의
패키지	`medirun_dispatch`
실행 위치	TurtleBot3 온보드 Raspberry Pi
목적	시리얼로 들어온 SOS 입력을 안전한 배송 미션 요청으로 변환
소유하지 않는 기능	Nav2 직접 제어, 급정지 판정, 도킹 제어, 통합 DB 전송

```
waypoint_file: config/waypoint_map.yaml
debounce_ms: 300
duplicate_window_sec: 3.0
queue_size: 5
mission_policy: QUEUE
battery_reject_threshold: 30.0
require_localization: true
require_payload: true
default_priority: 100
```

순서	검사·동작	실패 코드
1	`pressed=true`와 메시지 시각 유효성 검사	INVALID_EVENT
2	300ms 디바운스, 3초 중복 이벤트 제거	DUPLICATE
3	급정지·이동 허용·배터리·localization 검사	SAFETY_BLOCKED
4	AED 적재 상태 확인	PAYLOAD_MISSING
5	`button_id`를 `waypoint_map`에서 조회	UNKNOWN_BUTTON
6	현재 미션 유무·큐 용량·정책 확인	QUEUE_FULL / BUSY
7	`mission_id` 생성 후 로컬 DB에 호출 레코드 기록	LOG_WRITE_FAILED
8	`MissionRequest` 발행 (통합 DB 반영을 기다리지 않음)	PUBLISH_FAILED

```
BTN_01:
  zone: '개찰구 A'
  destination_key: 'gate_a'
  frame_id: 'map'
  pose: {x: 1.20, y: 0.80, yaw: 0.00}
BTN_02:
  zone: '승강장 1'
  destination_key: 'platform_1'
  frame_id: 'map'
  pose: {x: 3.50, y: -1.10, yaw: 1.57}
BTN_03:
  zone: '대합실 중앙'
  destination_key: 'hall_center'
  frame_id: 'map'
  pose: {x: 2.10, y: 2.40, yaw: 3.14}
```

3.2 safety_manager

항목	정의
입력	/medirun/emergency_stop, /scan, /arduino/ultrasonic, /amcl_pose, /battery_state, 상위 /cmd_vel
출력	/medirun/safety/status, 최종 /cmd_vel, /medirun/audio/say
타이머	10Hz 판정 루프

```
front_clear_distance: 0.40
low_obstacle_distance: 0.15
clear_hold_sec: 3.0
auto_resume_max_count: 3
battery_return_threshold:  20.0
max_linear_velocity: 0.05
max_angular_velocity: 0.7
```

주요 함수: `gate_cmd_vel()`, `on_estop()`, `evaluate_resume_conditions() -> (bool, blocking_list)`, `request_resume()`, `log_decision()`

> **예외 처리:** 초음파 데이터를 2초 이상 미수신하면 재출발 불가 판정을 유지한다(`SENSOR_TIMEOUT`). 센서가 침묵하는 것을 안전하다는 뜻으로 해석하지 않는다.

3.3 payload_monitor

항목	정의
입력	/arduino/hall, /medirun/mission/status

항목	정의
출력	`/medirun/payload/state`, `/medirun/system/event`, 복귀 요청
파라미터	`debounce_ms: 500`, `return_delay_sec: 15.0`, `no_pickup_notice_sec: 60.0`, `no_pickup_timeout_sec: 180.0`

동작: 상태 변화가 0.5초 이상 유지되면 유효 변화로 처리한다. ARRIVED 상태에서 `aed_present`가 true→false로 바뀌면 반출 이벤트를 기록하고 15초 뒤 복귀 미션을 요청한다. 미반출 상태에서는 1분마다 안내 음성을 1회 출력하고, 3분이 지나면 AED를 실은 채 복귀한다. IDLE 상태에서 false가 지속되면 대시보드에 적재 요청 경고를 표시한다.

3.4 미션 상태 머신

상태	의미	주요 전이
IDLE	호출 대기	SOS 수신 → VALIDATING
VALIDATING	호출·환경·좌표·적재 검증	정상 → DISPATCHED / 실패 → REJECTED
DISPATCHED	미션 확정·Nav2 목표 준비	목표 수락 → NAVIGATING
NAVIGATING	목적지 주행(경고 음성·경광 LED)	성공 → ARRIVED / 실패 → RETRY_WAIT / 배터리 20% 미만 → RETURNING_HOME
RETRY_WAIT	비용지도 초기화 후 재시도	2회 이내 → NAVIGATING / 초과 → FAILED
ARRIVED	도착 알림·119 안내·반출 대기	반출 감지 후 15초 또는 미반출 3분 경과 → RETURNING_HOME
RETURNING_HOME	홈 위치 이동	도착 → DOCKING / 실패 → FAILED
DOCKING	ArUco 정렬·파킹	성공 → DOCKED → IDLE / 실패 3회 → FAILED
QUEUED	활성 미션 뒤 대기	순서 도달 → VALIDATING

AURIX

4. 데이터 · 로그 설계

저장 원칙

모든 기록은 로봇 내부 PostgreSQL에 먼저 쓰고, 동기화 워커가 통합 DB로 옮긴다. 기록 성공 여부는 로컬 쓰기로 확정되며 통합 DB 도달을 기다리지 않는다. 연결이 정상이면 수 초 이내에 반영되고, 끊겨 있으면 로컬에 누적됐다가 복구 시 시간순으로 전송된다.

4.1 저장 구조

계층	위치	내용
1차	로봇 내부 PostgreSQL	미션·이벤트·주행 기록·센서 스냅샷 (`synced` 플래그 포함). 원본이자 단절 시 버퍼
2차	원격 PC 통합 PostgreSQL	전체 이력 장기 보관, 대시보드 이력 조회 대상

`ROBOT_SESSION`의 `id`를 `mission_id`로 사용하고, `SYSTEM_EVENT`가 이를 참조해 호출부터 복귀까지의 이력을 단일 조회한다.

```
ROBOT_SESSION(id, robot_id, button_id, destination_key, status,
              requested_at, dispatched_at, arrived_at,
              payload_removed_at, returned_at, completed_at,
              result_code, synced)

SYSTEM_EVENT(event_id, mission_id, robot_id, severity,
             event_type, message, occurred_at, synced)
```

4.2 이벤트 규칙

event_type	severity	필수 메시지
SOS_RECEIVED	WARN	button_id, zone, call_x, call_y, occurred_at
PAYLOAD_MISSING	ERROR	mission_id, checked_at
MISSION_DISPATCHED	INFO	mission_id, destination
NAVIGATION_RETRY	WARN	attempt, Nav2 result, costmap_cleared
EMERGENCY_STOP	CRITICAL	source(GUI), operator, reason
SAFETY_AUTO_RESUME	WARN	resume_count, 충족·미충족 조건 목록
LOW_OBSTACLE_STOP	WARN	distance_cm, side
MISSION_ARRIVED	INFO	duration, position_error

event_type	severity	필수 메시지
GUIDANCE_119_PLAYED	INFO	mission_id, played_at
PAYLOAD_REMOVED	INFO	mission_id, removed_at
RETURN_TRIGGERED	INFO	source(payload/timeout/battery/manual)
BATTERY_RETURN	WARN	battery_percent, interrupted_mission
DOCKING_RESULT	INFO/ERROR	attempts, marker_id, offset
CONNECTION_LOST	ERROR	peer, last_seen
CONNECTION_RESTORED	INFO	downtime_sec, synced_records
LOG_WRITE_FAILED	CRITICAL	target, error, fallback_path
MISSION_FAILED	ERROR	error_code, last_state, return_attempted
LOG_RETENTION	INFO	deleted_rows, pending_rows, disk_usage
MISSION_COMPLETED	INFO	result, completed_at

4.3 식별자 · 시간

항목	규칙
식별자 형식	`mission_id: MR-YYYYMMDD-NNN, event_id: EV-YYYYMMDD-HHMMSS-NNN`
DB 시간	`timestamp`로 저장하고 화면에는 Asia/Seoul 기준으로 표시한다
지연 분석	ROS 메시지 Header stamp와 기록 시각을 함께 저장해 처리 지연을 분석한다
동기화 시각	로컬 기록 시각과 통합 DB 반영 시각을 모두 남겨 SR-P-011 측정에 사용한다
설정 변경	`waypoint_map` 변경 시 version과 생성 시각을 남긴다

AURIX